



信息学奥林匹克竞赛

Olympiad in Informatics

动态规划—斜率优化

陈 真

上海市实验学校

动态规划优化（基础）

优化4: DP+斜率优化

考虑一类状态转移方程：

$$DP_i = \min/\max\{DP_j + A_i B_j + C_i + D_j\}$$

其中 DP_i , A_i , B_i 是和 i 有关的函数, DP_j , B_j , D_j 是和 j 有关的函数,

如果没有 $A_i B_j$,那么原式可以转为: $DP_i - C_i = \min/\max\{DP_j + D_j\}$ 可直接用单调队列, 维护一个最小/最大的 $DP_j + D_j$ 的单调队列即可。因 $A_i B_j$ 无法使用单调队列优化。

【转换】：

$(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$, 寻找决策点 j 转移给 i , j 便是里面取得最小值得那个点。

类比一次函数 $y=kx+b \rightarrow \mathbf{b=y-kx}$, 将上述得四个值对应转移方程得四个值:

$$b_i = DP_i - C_i \quad y_i = DP_j + D_j \quad k_i = -A_i \quad x_j = B_j$$

找决策点 j , 而 i 固定, A_i 是固定的, 最小化 DP_i 等价于最小化 b 截距。

动态规划优化 (基础)

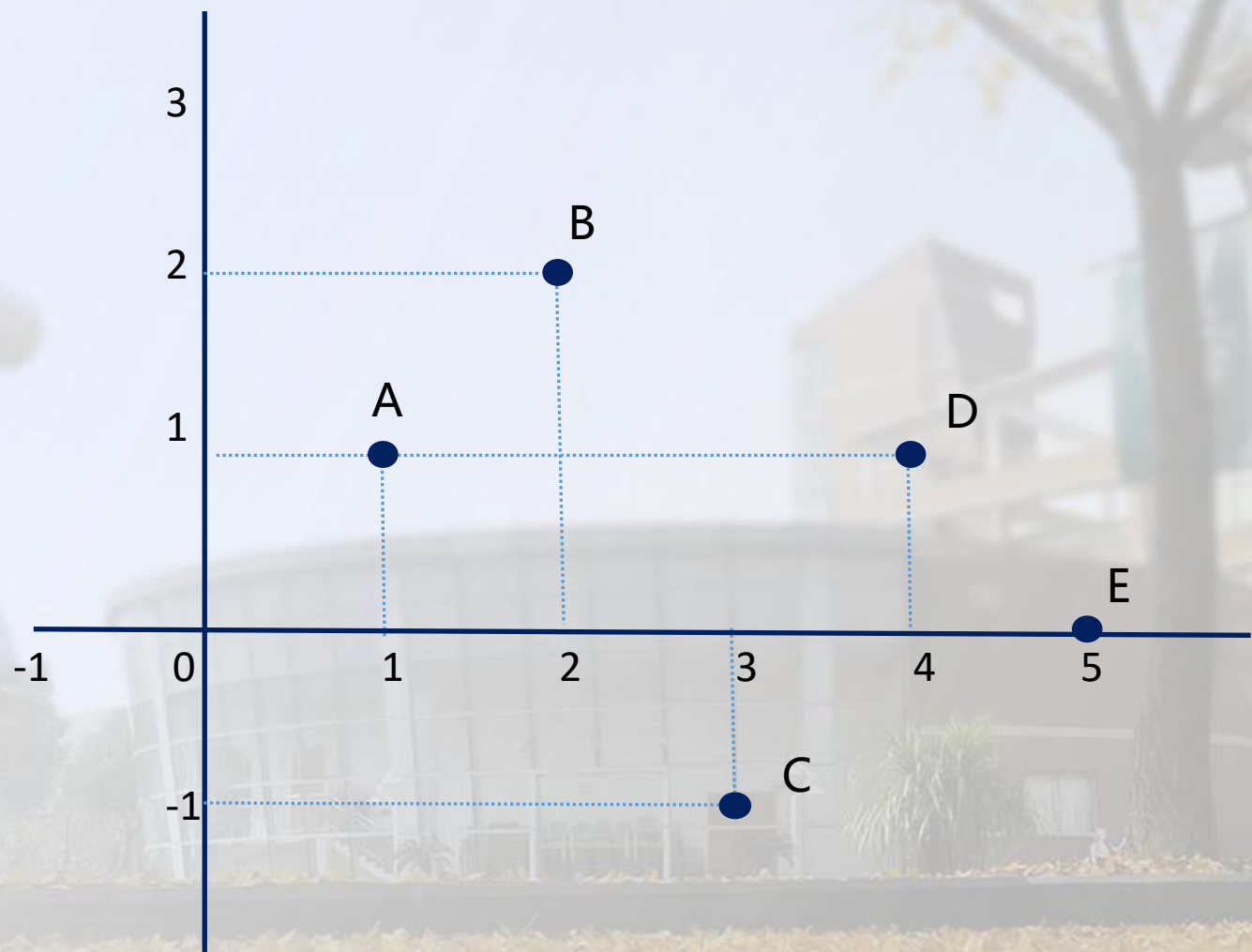
优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_j - k_i x_j\}$$

- 固定一个 i , 那么 k_i 固定, 设 $len=r-l+1$, 对于 $[l, r]$ 之间的所有 j , 对应于一个大小为 len 点集 (x_j, y_j) 。如图五个点且 x_j 严格递增。
- 一个过 (x_j, y_j) 且斜率为 k_i 的直线, 是唯一存在。
- len 个点对应于 len 条直线。

由右图观察, 截距最小的直线CE来进行转台转移。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

假设场景: x_j 递增, k_i 递增的情况。

一般的题目中, 这个 $[l, r]$ 区间, 往往取的是 $[1, i-1]$

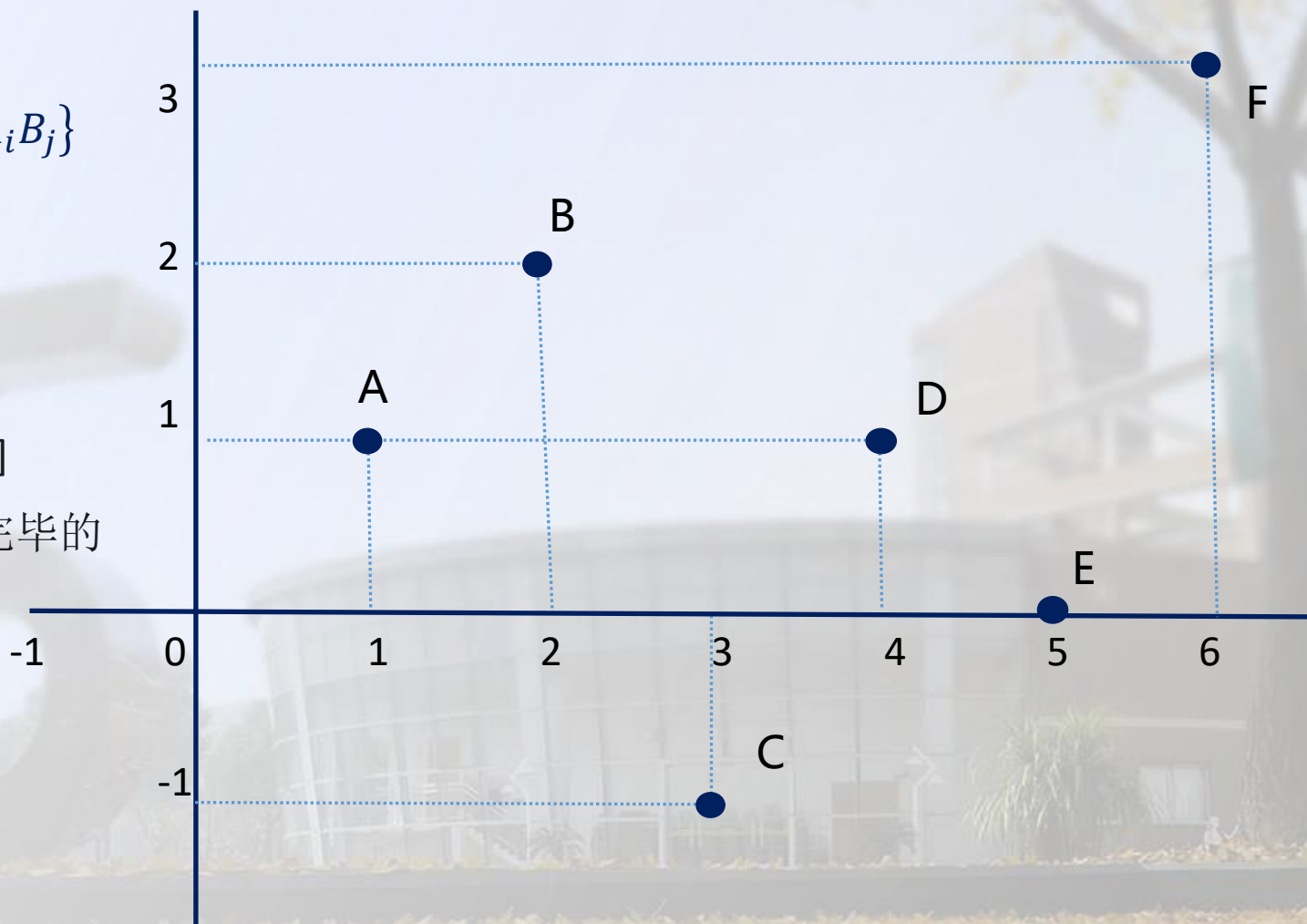
这个点集的大小只会被增加不会减小, 而计算完毕的

$dp[i]$, 又往往和 y_i 有关,

所以我们在计算 $dp[i+1]$ 的时候,

就需要加入 (x_i, y_i) 到这个点集中。

•新加入F点。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

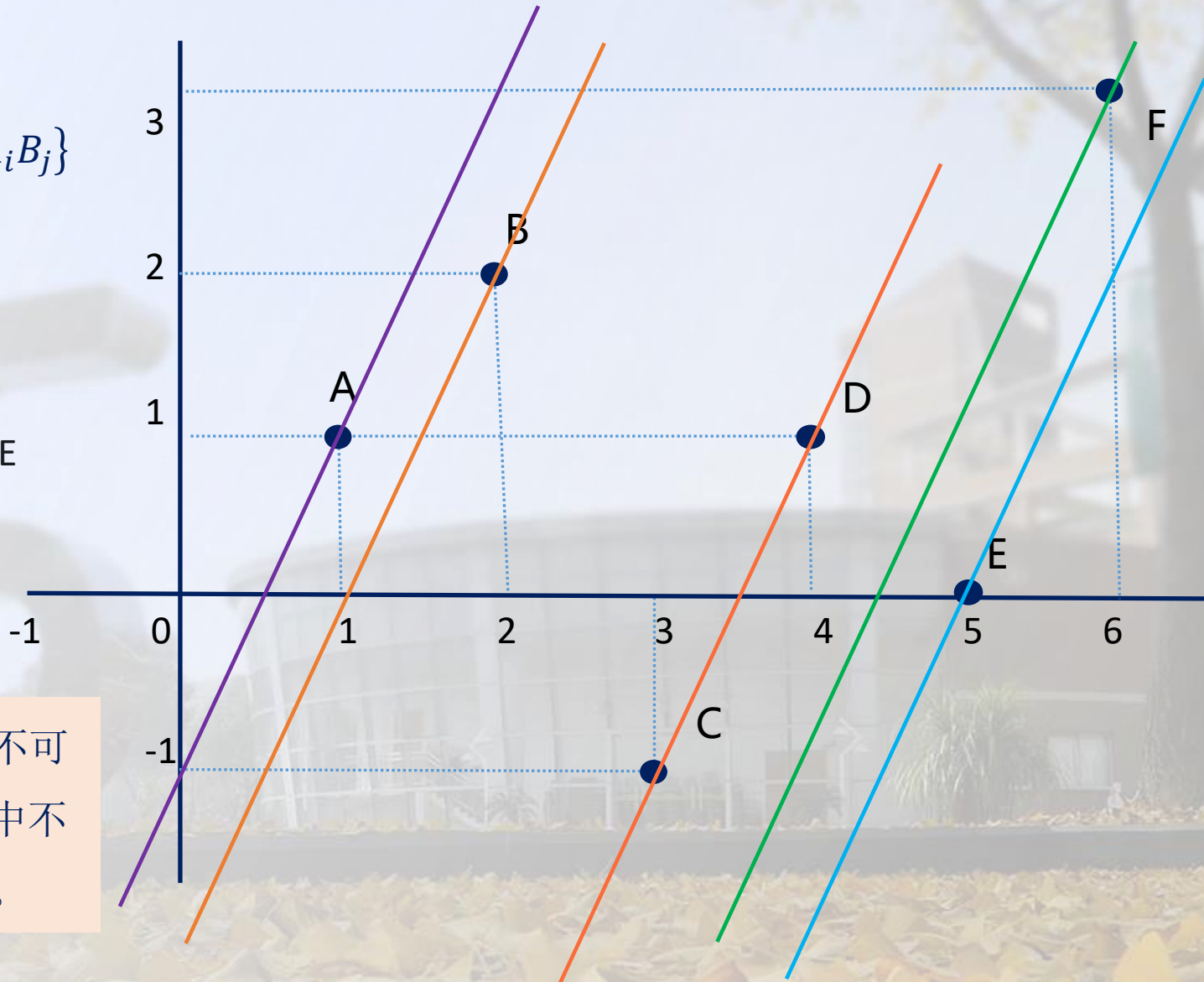
$$b_i = \min\{y_i - k_i x_j\}$$

•新加入F点。

此时斜率 k_i 也增大, 假设 $k_i = 2$, 可以看到此时E点所在直线对应的截距是最小的。

状态转移, 转换为“点集”中选取最小截距。

【思考】 分析不论斜率 k_i 值为多少, B、D点都不可能当作转移状态的前驱, 如果能够剔除点集中不可能转移的点, 计算量减小, 复杂度得到优化。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

假设有两个点A,B, 设二者所在直线的斜率为 k_{AB} .

如果过AB的直线在旋转, 斜率随之变化, 旋转过程中有时A点优, 有时B点优。

旋转的边界值: $k_i = k_{AB}$;

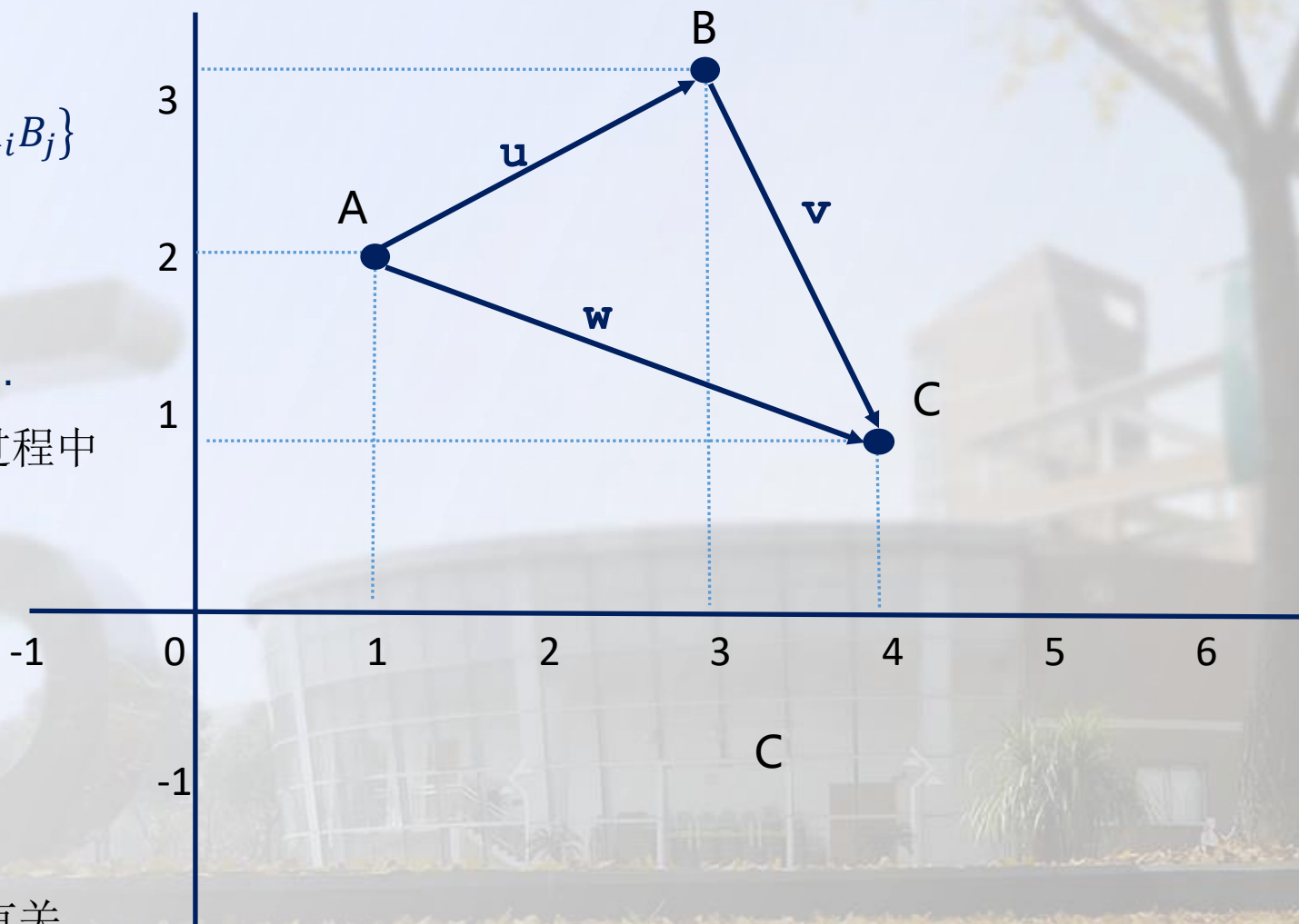
斜率不存在是 (平行y轴)

旋转经过这两个边界时, 最优点发生改变。

对于AC来说, B点时一个上凸点。

总结可得最优点, k_i 与 k_{AB}, k_{BC}, k_{AC} 的大小关系有关。

目前: $k_{BC} < k_{AC} < 0 < k_{AB}$, 经过这三个点的直线在旋转, 斜率随之变化: $k_i \rightarrow -\infty$ 向 $k_i \rightarrow +\infty$, 斜率一直增大。



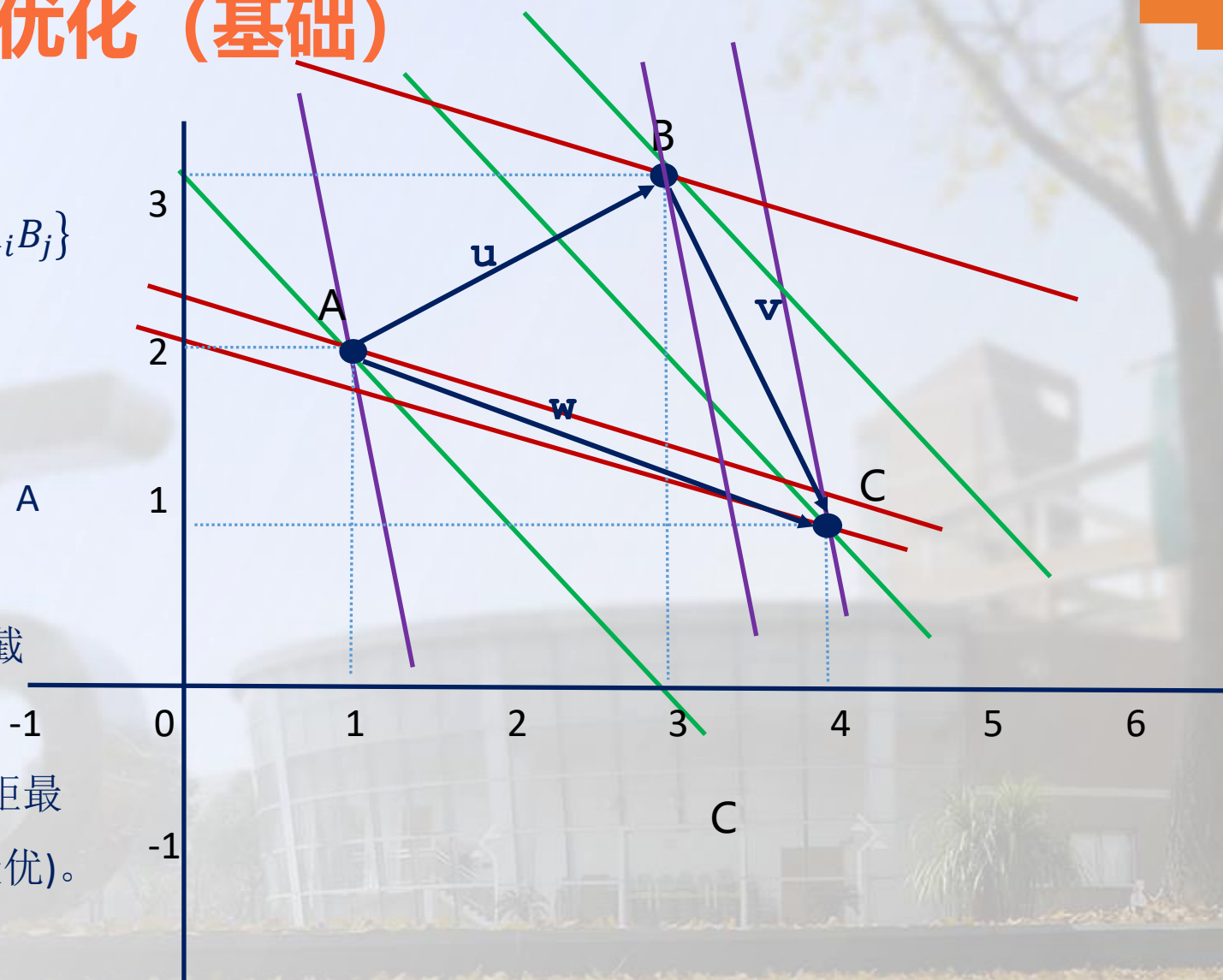
动态规划优化 (基础)

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

- 当时 $k_i \leq k_{BC}$, A点所在直线(紫色)截距最小, A点最优。
- 当时 $k_{BC} \leq k_i \leq k_{AC}$, A点所在直线(绿色)截距最小, A点最优。
- 当时 $k_{AC} \leq k_i \leq 0$, C点所在直线(红色)截距最小(划分不同斜率区间, 重合点A,C同时取到最优)。



动态规划优化（基础）

优化4: DP+斜率优化

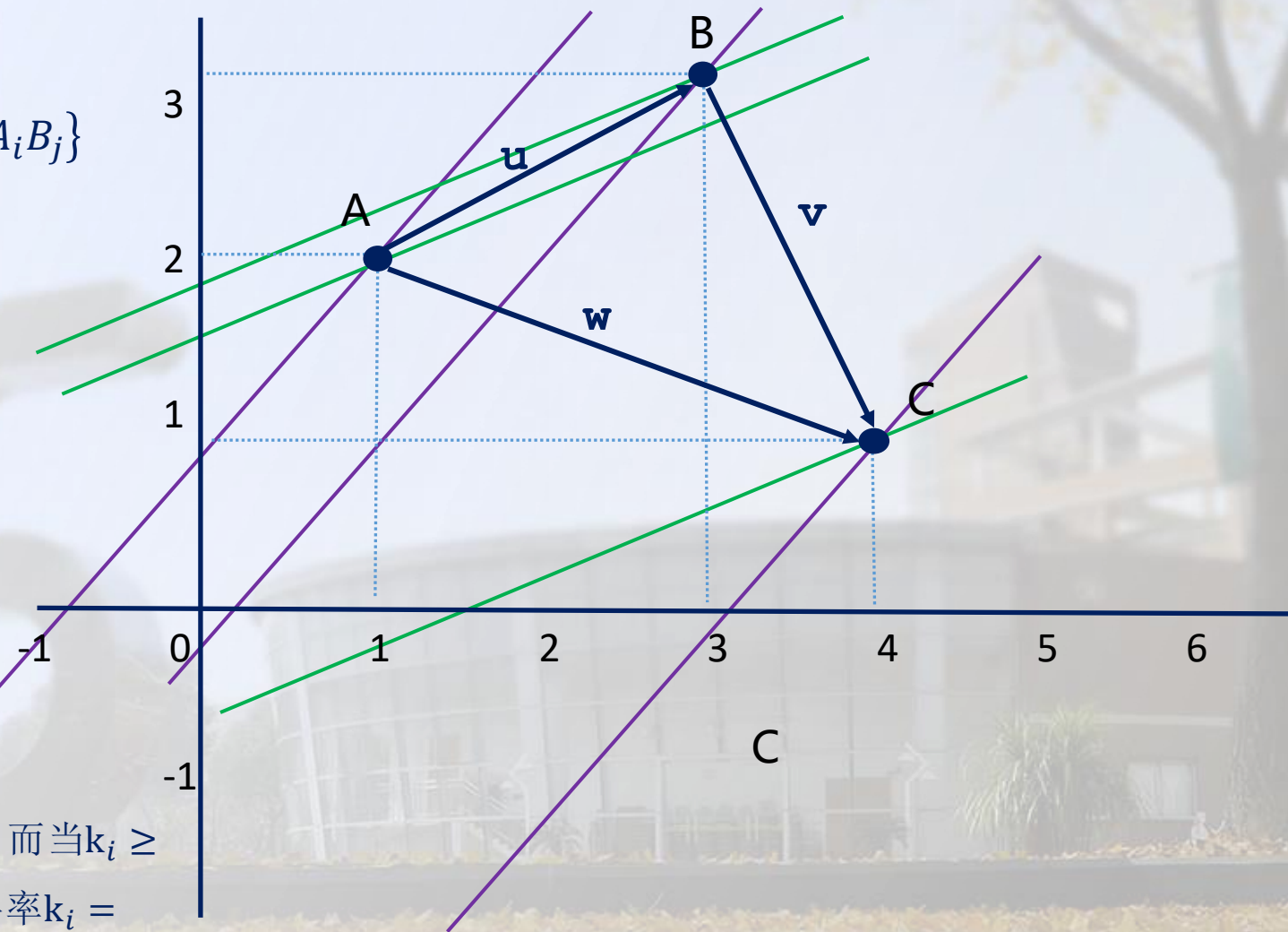
求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

- 当 $0 \leq k_i \leq k_{AB}$, C点依旧是最优点。（绿色）
- 当 $k_i \geq k_{AB}$, C点是最优点。（紫色）

总结: 1. 无论取什么值, 都不可能成为最优点。所以这样的“上凸点”对于状态转移来说, 是冗余点, 舍弃不参与计算。

2. 因为 k_i 是递增, 当 $k_i < k_{AC}$ 时, 最优点一直都是A点。而当 $k_i \geq k_{AC}$ 时, 最优点一直都是C点。考虑两个点的情况, 斜率 $k_i = k_{AC}$ 就是最优点的分界线。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

【思路】 维护一个下凸包就可以减少大量的无用计算，在凸包上找到一个点 P_i ，其前后的点分别为 P_{i-1} ， P_{i+1} 满足 $k_{P_{i-1}P_i} \leq k_i \leq k_{P_iP_{i+1}}$ 。这样的话，因为 $k_{P_{i-1}P_i} \leq k_i$ ，所以 P_i 比它前面的点都优秀，又，所以又 $k_i \leq k_{P_iP_{i+1}}$ 后面的点都优秀，综述，它就是全局最优点，也就是我们状态转移的前驱。

因为 k_i 是递增，二分查找最优点，其一直右移具有单调性。可使用单调队列，当头部满足 $k < k_i$ 时候就出队，时刻维护头部的 $k > k_i$ 。每次取到的都是最优点。另外新加入的点，也就相当于放入单调队列的尾部，尾部进行去除上凸点的操作。

动态规划优化（基础）

【题目05】任务安排（P2365）

【题目描述】

n 个任务排成一个序列在一台机器上等待完成（顺序不得改变），这 n 个任务被分成若干批，每批包含相邻的若干任务。

从零时刻开始，这些任务被分批加工，第 i 个任务单独完成所需的时间为 t_i 。在每批任务开始前，机器需要启动时间，而完成这批任务所需的时间是各个任务需要时间的总和（同一批任务将在同一时刻完成）。

每个任务的费用是它的完成时刻乘以一个费用系数 f_i 。请确定一个分组方案，使得总费用最小。

【输入格式】

第一行一个正整数 n 。

第二行是一个整数 s 。

下面 n 行每行有一对数，分别为 t_i 和 f_i ，表示第 i 个任务单独完成所需的时间是 t_i 及其费用系数 f_i 。

【输出格式】

一个数，最小的总费用。

【样例输入 #1】 【样例输出 #1】

对于 100% 的数据， $1 \leq n \leq 5000$ ， $0 \leq s \leq 50$ ， $1 \leq t_i, f_i \leq 100$ 。

5
1
153

【样例解释】如果分组方案是：

{1, 2}, {3}, {4, 5}, 则完成时间分别为{5, 5, 10,
14, 14}, 费用 $C=15+10+30+42+56$,总费用就是153。

1 3
3 2
4 3
2 3
1 4

动态规划进阶应用

【题目05】任务安排（P2365）

【分析】设 $dp[i][j]$ 表示前 i 个任务分成 j 组完成的最小代价。

$1, 2, 3, \dots, k-1, k, k+1, \dots, i$

$$dp[i][j] = \min_{1 \leq k \leq i} \{ dp[k][j-1] + (S \times j + \sum_{x=1}^i T_x) \times \sum_{x=k+1}^i f_x \}$$

$$\text{令 } \text{sum}T_i = \sum_{k=1}^i T_k, \quad \text{sum}C_i = \sum_{k=1}^i f_k$$

$$dp[i][j] = \min_{1 \leq k \leq i} \{ dp[k][j-1] + (S \times j + \text{sum}T_i) \times (\text{sum}C_i - \text{sum}C_k) \}$$

答案: $\sum_{j=1}^n \min\{dp[n][j]\}$ 时间复杂度: $O(n^3)$

```
15 | for(int i=1;i<=n;i++)
16 |     dp[i][1]=(sumT[i]+s)*sumC[i]; //初始化
17 | for(int i=2;i<=n;i++)
18 |     for(int j=2;j<=i;j++)
19 |         for(int k=1;k<i;k++)
20 |             dp[i][j]=min(dp[i][j],dp[k][j-1]+(s*j+sumT[i])*(sumC[i]-sumC[k]));
```

思考：根据数据范围分析会出现超时和超内存，需要进一步**优化**。

动态规划进阶应用

【题目05】任务安排（P2365）

【优化】 $dp[i][j]$ 中 j 只与以后任务生产的代价有关，**降维**。

$1, 2, 3, \dots, k-1, k, k+1, \dots, i$

设 $dp[i]$ 表示前 i 个任务的最小费用总和。

依然考虑将任务枚举前 k 个， $[k+1, i]$ 分为一批。前 k 个任务完成最小代价 $dp[k]$ 。

计算代价分为两部分： $\sum_{j=1}^i T_j \times \sum_{j=k+1}^i p_j$ 和 $S \times \sum_{j=k+1}^i p_j$ 。所以可得转移方程：

$$dp[i] = \min_{0 \leq k < i} \{ dp[k] + \sum_{j=1}^i T_j \times \sum_{j=k+1}^i p_j + S \times \sum_{j=k+1}^i p_j \}$$

$$\text{令 } \text{sum}T_i = \sum_{k=1}^i T_k, \quad \text{sum}C_i = \sum_{k=1}^i f_k$$

$$dp[i] = \min_{0 \leq k < i} \{ dp[k] + S \times (\text{sum}C_n - \text{sum}C_k) + \text{sum}T_i \times (\text{sum}C_i - \text{sum}C_k) \}$$

初始值： $dp[0]=0$ 答案： $dp[n]$ 时间复杂度： $O(n^2)$

```
16 dp[0]=0;
17 for(int i=1;i<=n;i++)
18     for(int k=0;k<i;k++)
19         dp[i]=min(dp[i],dp[k]+S*(sumC[n]-sumC[k])+sumT[i]*(sumC[i]-sumC[k]));
20 cout<<dp[n]<<"\n";
```

动态规划进阶应用

【题目06】任务安排2 (P10979, P2365 强化版)

数据范围：对于 100% 的数据， $1 \leq n \leq 3 \times 10^5$ ， $0 \leq s \leq 2^8$ ， $1 \leq t_i \leq 2^8$ ， $0 \leq f_i \leq 2^8$ 。

$$dp[i] = \min_{0 \leq k < i} \{dp[k] + S \times (\text{sum}C_n - \text{sum}C_k) + \text{sum}T_i \times (\text{sum}C_i - \text{sum}C_k)\}$$

$$dp[i] = dp[k] + S \times (\text{sum}C_n - \text{sum}C_k) + \text{sum}T_i \times (\text{sum}C_i - \text{sum}C_k)$$

$$dp[i] = dp[k] + S \times \text{sum}C_n - S \times \text{sum}C_k + \text{sum}T_i \times \text{sum}C_i - \text{sum}T_i \times \text{sum}C_k$$

$$y = x \cdot k + b$$

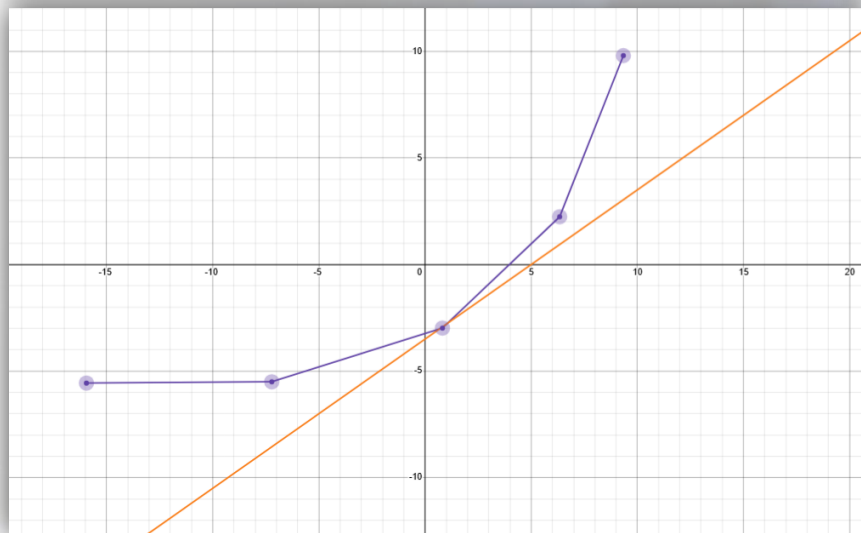
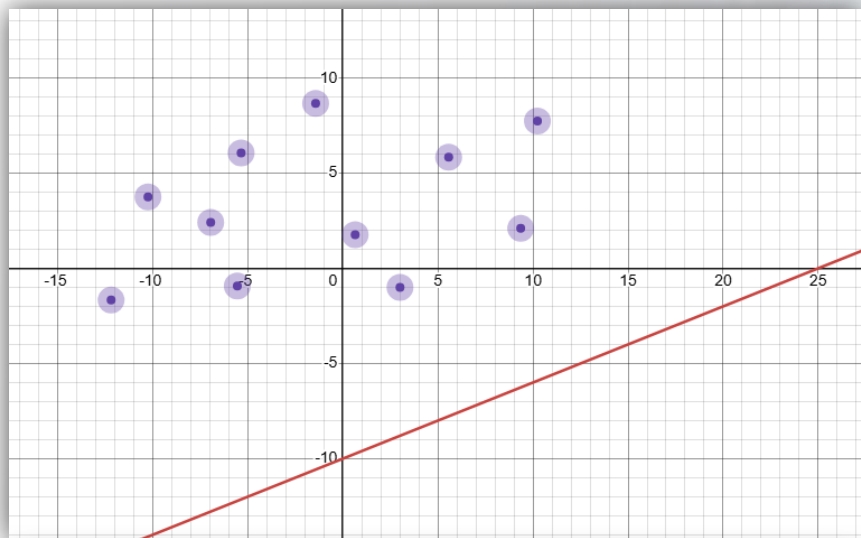
斜率优化

$$dp[k] = \text{sum}C_k (S + \text{sum}T_i) + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

横坐标 $\text{sum}C_k$ ，纵坐标 $dp[k]$ ，斜率为： $S + \text{sum}T_i$ ，截距为： $dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$

动态规划进阶应用

【题目06】任务安排2（P10979, P2365 强化版）



$$dp[k] = \text{sum}C_k(S + \text{sum}T_i) + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

$$y = x \cdot k + b$$

当*i*固定时，直线的斜率也是固定的，每当直线经过一点可以得到一个截距。
当截距最小时， $dp[i]$ 取到最小值。

设1, 2两点的连线的斜率 k_1 ，2, 3两点的连线斜率为 k_2 ，进行决策的直线斜率为 K ，一个点是最优解，则 $k_1 \leq K \leq k_2$ 。

😞 由于 $0 \leq k < i$ ，当*i*增加时会有新的点加入， $\text{sum}C_i$ 具有单调性。根据本题的情况只有下凸壳中的点才能可能最优，下凸壳中线段斜率单调递增，可用单调队列维护。

$\text{sum}T_i$ 是单调递增（斜率），凸包最右边的未选上的点不可能被选上，所以将其弹出队列，队列最右端的点即为答案。

动态规划进阶应用

【题目06】任务安排2 (P10979, P2365 强化版)

核心代码实现

```
14  memset(dp,0x3f,sizeof dp);
15  dp[0]=0;l=r=1;
16  q[1]=dp[0];
17  for(int i=1;i<=n;i++){
18      while(l<r&&(dp[q[l+1]]-dp[q[l]])<=(s+sumT[i])*(sumC[q[l+1]]-sumC[q[l]]))l++;
19      //最右侧两点形成线段的斜率小于当前决策斜率,右侧点不可能再被使用,弹出队列。
20      int k=q[l];
21      dp[i]=dp[k]+s*(sumC[n]-sumC[k])+sumT[i]*(sumC[i]-sumC[k]);
22      while(l<r&&(dp[q[r]]-dp[q[r-1]])*(sumC[i]-sumC[q[r]])>=(dp[i]-dp[q[r]])*(sumC[q[r]]-sumC[q[r-1]]))r--;
23      //维护单调队列中斜率的单调性
24      q[++r]=i;
25  }
```

【题目07】[SDOI2012] 任务安排 (P5785)

数据范围：对于 100% 的数据, $1 \leq n \leq 3 \times 10^5$, $0 \leq s \leq 2^8$, $1 \leq t_i \leq 2^8$, $0 \leq f_i \leq 2^8$ 。



数据范围：对于 100% 的数据, $1 \leq n \leq 3 \times 10^5$, $0 \leq s \leq 2^8$, $1 \leq |T_i| \leq 2^8$, $0 \leq C_i \leq 2^8$ 。

动态规划进阶应用

【题目07】 [SDOI2012] 任务安排 (P5785)

$$y = x \cdot k + b$$

$$dp[k] = \text{sum}C_k (S + \text{sum}T_i) + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

分析：【题目06】到【题目07】变换在于 $1 \leq |T_i| \leq 2^8$ 。 $\text{sum}T_i$ 将不具备单调性

$$dp[k] = (S + \text{sum}T_i) \times \text{sum}C_k + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

由于单调性不满足，因此不能弹出队头，要维护整个凸壳。凸壳具有下凸性，最优决策点的斜率 $\text{sum}T[i]+s$ 满足：

$K_{\text{左侧}} < \text{sum}T_i + S < K_{\text{右侧}}$ ，可借助二分寻找决策点。

思路：利用一个单调栈维护整个下凸壳，单调栈内的斜率是递增的，每次二分出最后一个比决策直线斜率小的线段，其右端点即为答案。

动态规划进阶应用

【题目07】[SDOI2012] 任务安排 (P5785)

核心代码实现

```
6 int search(int i){
7     int l=1,r=top,ans=0;
8     while (l<=r){
9         int mid=(l+r)>>1,a=sta[mid-1],b=sta[mid];
10        if(dp[b]-dp[a]<=(sumT[i]+s)*(sumC[b]-sumC[a]))
11            l=mid+1,ans=mid;
12        else
13            r=mid-1;
14    }
15    return sta[ans];
16 }
17 signed main(){
18     cin>>n>>s;
19     for(int i=1;i<=n;i++)
20         cin>>t[i]>>p[i];
21     for(int i=1;i<=n;i++){
22         sumT[i]=sumT[i-1]+t[i];
23         sumC[i]=sumC[i-1]+p[i];
24     }
25     memset(dp,0x3f,sizeof dp);
26     sta[top=1]=dp[0]=0;
27     for (int i=1;i<=n;i++){
28         int k=search(i);
29         dp[i]=dp[k]+s*(sumC[n]-sumC[k])+sumT[i]*(sumC[i]-sumC[k]);
30         while(top>1&&(dp[sta[top]]-dp[sta[top-1]]*(sumC[i]-sumC[sta[top]])>=(dp[i]-dp[sta[top]]*(sumC[sta[top]]-sumC[sta[top-1]])))
31             top--;
32         sta[++top]=i;
33     }
34     cout<<dp[n]<<'\\n';
```

动态规划进阶应用

推荐题目

序号	题目	备注
1	[NOIP 2018 普及组] 摆渡车 (P5017)	
2	[ZJOI2007] 仓库建设 (P2120)	两项单调
3	[HNOI2008] 玩具装箱 (P3195)	两项单调
4	[USACO08MAR] Land Acquisition G (P2900)	两项单调
5	[APIO2014] 序列分割 (P3648)	两项单调
6	[CEOI 2004] 锯木厂选址 (P4360)	两项单调
7	[APIO2010] 特别行动队 (P3628)	两项单调
8	[SDOI2016] 征途 (P4072)	两项单调
9	[CEOI 2017] Building Bridges (P4655)	$X(j)$ 与 $k0[i]$ 均不单调)
10	高速公路 (P3394)	$X(j)$ 单调 $k0[i]$ 不单调, 树上转移)
11	[NOI2007] 货币兑换 (P4027)	($X(j)$ 与 $k0[i]$ 均不单调)
12	[NOI2014] 购票 (P2305)	($X(j)$ 单调 $k0[i]$ 不单调。)
13	[NOI2016] 国王饮水记 (P1712)	两项单调